

AJ 아바타 학습 시스템 가이드

회원의 AI 아바타가 게임을 스스로·사람에게·개발자에게 배워 점점 잘하게 되는 구조를 방식별로 정리한 기술 문서입니다.

Vibrexcup 내부 문서 · 2026 · 예시는 브레이커(벽돌깨기) 계열 게임 기준

공통 토대 — 상태·행동·보상

모든 학습은 게임이 노출하는 3요소를 재료로 씁니다. 이는 강화학습의 기본 틀과 동일합니다.

```
VIBREX_GAME = {
  state()  → { ballX, ballY, ballDx, ballDy, paddleX, paddleW, attached, score, lives,
  inputs   → left / right / action(발사)
  phase()  → title | playing | over | cleared
  progress() → 0 ~ 1
}
보상(Reward) = 점수 ↑ (+) · 클리어 (+) · 게임오버 (-)
```

왜 DQN/PPO(신경망 강화학습)를 쓰지 않았나 — 회원이 만드는 게임이 수천 개라, 게임 1개당 GPU로 신경망을 수만 판 학습시키는 것은 비용·인프라상 불가능합니다. 대신 **사람이 읽을 수 있는 규칙 + 정석 휴리스틱 + 가벼운 진화(CEM) + LLM 최적화**를 조합했습니다. 효과는 유사하면서 비용이 거의 0이고, "무엇을 배웠는지"가 사람에게 그대로 보입니다.

1 AI 직접 플레이 (오토파일럿)

게임을 실제로 조작하는 실행 엔진. 두 종류를 자동 선택합니다.

완성형 봇 (정석 알고리즘)

새 템플릿 게임에 내장. 브레이커는 공 낙하점 물리 예측 (벽 반사까지 계산), 테트리스는 모든 회전×열 착지 시뮬레이션 후 구멍·높이 평가.

폴백 봇

매니페스트가 없으면 키 입력 + 화면 클릭 + 아래에서 학습한 규칙(조건→행동)을 실행.

실력은 `setSkill(0~1)` 로 조절 — 낮으면 반응이 느리고 실수, 높으면 정밀. 이 실력은 ② **기본기 학습**으로 0.3 → 1.0까지 단계 해 제됩니다.

2 템플릿 기본기 커리큘럼

템플릿이 미리 보유한 정석 플레이를, 회원 AI가 **2판 + 1시간 간격**으로 한 단계씩 학습합니다. 정석 힌트를 이 게임의 `state` 에 맞 는 실행 규칙으로 번역(Haiku).

브레이커 커리큘럼 — 1. 공 따라가기 → 2. 낙하점 예측 → 3. 가장자리로 각도 조절 → 4. 옆 통로 뚫기

1단계 “공 따라가기” 컴파일 결과:

```
{ "cond": "s.ballDy > 0 && s.ballX > s.paddleX + 8", "action": "right", "hold": 80 }  
// 공이 내려오는 중이고 패들보다 오른쪽 → 오른쪽으로 80ms 이동
```

단계가 오를수록 규칙이 정교해지고 봇 실력(botSkill)도 함께 해제됩니다. 관리자·개발자가 단계를 추가할 수 있습니다.

3 프롬프트 코칭 (말로 가르치기)

게임 참여 중 채팅으로 가르치면 실행 규칙으로 컴파일됩니다. 이 규칙은 **게임별·나만의 것**이라 다른 회원 AI에게는 가지 않습니다.

내 입력: “공이 왼쪽 끝으로 가면 미리 왼쪽 벽에 붙어”

```
{ "cond": "s.ballX < 60 && s.ballDx < 0", "action": "left", "hold": 200, "why": "왼쪽 벽
```

AJ 응답: “왼쪽 벽 선점 규칙 추가했어 (학습 v15 · 규칙 3개)”

4 내 플레이 모방 (사람 조작 관찰)

내가 직접 플레이하면 0.2초마다 (**상태, 눌린 키, 탭 위치**)를 기록합니다. 이를 통계로 요약해 “사람은 이럴 때 이렇게 하더라”를 규 칙으로 만듭니다.

[인간 데모 요약]

- right 누름 (32/120 샘플): ballX 평균 210 (안 누를 때 140)
- action 누름 (8/120): attached=1, paddleX≈ballX 일 때
- 화면 탭 위치: 우하 12회, 좌하 8회

→ “공이 오른쪽일 때 오른쪽으로 갔다”는 패턴을 학습. AJ 학습 탭의 ‘내 플레이로 학습’ 버튼으로 즉시 반영할 수도 있습니다.

5 자기 반성 + 파라미터 진화 (알아서 성장)

3판마다 최근 점수를 스스로 돌아봅니다.

- 규칙이 있는 게임: “평균 X점, 더 잘하려면?”을 자문해 규칙을 조금씩 수정 (LLM as optimizer).
- 규칙을 못 만드는 게임: `reactionMs·randomness·botSkill` 등 숫자 파라미터를 변이 (가벼운 진화 / CEM).
- 안전장치: 새 버전 평균이 역대 최고 평균의 85% 미만이면 **잘되던 규칙으로 자동 복귀** (알파고식 “더 나은 정책만 채택”).

자기 반성 · 파라미터 진화(변이 시도) - `reactionMs 30.56 · randomness 0.01 · botSkill 0.35 · v15`

6 개발자 가이드 (제작자가 가르치기)

게임 제작자가 내정보 > 내 게임 > 학습에서 자기 게임의 정석을 단계로 등록하면, 그 게임을 하는 모든 회원의 AI가 ②처럼 시간차로 배웁니다. (내 코칭·모방은 나만 보유, 개발자 가이드·기본기는 공용)

전체 흐름

AI 플레이(①) → 판 끝날 때마다 에피소드·점수 기록 → 3판마다 자기 반성(⑤)



| 실행 정책 = 기본기(②) + 내 코칭(③) + 내 플레이 모방(④) + 개발자 가이드(⑥)

| 실력(botSkill) = 기본기 진행도만큼 완성형 봇 해제



모든 학습은 (게임 × 회원) 단위로 저장 → 내정보 > AJ 학습 에서 확인

학습 기록에서 보이는 라벨

- AI 플레이
- 기본기
- 프롬프트 코칭
- 내 플레이 모방
- 자기 반성
- 개발자 가이드
- 복귀

방식	누가	공유 범위	기록 시점
AI 직접 플레이	AI	나	매 판 종료
템플릿 기본기	플랫폼	모두(단계별)	2판+1시간마다 1단계
프롬프트 코칭	나(채팅)	나	가르칠 때
내 플레이 모방	나(플레이)	나	75샘플(≈15초)마다
자기 반성/진화	AI	나	3판마다
개발자 가이드	제작자	그 게임의 모두	가이드 등록 시

중요한 전제 조건

학습은 게임이 **표준 상태 정보(매니페스트: `state()` · `inputs`)**를 노출해야 동작합니다. 2026-08-19 표준 도입 이후 만들어진 게임과 그 이후의 템플릿 게임에서 유효합니다. 그 이전 게임·외부 링크 게임은 관찰할 데이터가 없어 ① **AI 플레이 (범용 봇)**만 가능하고 ④⑤ 학습 데이터는 쌓이지 않습니다.